Pods: resource object:

```
apiversion: V1
Kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

Define a Service:

a service is a object like Pod.

resource object.

```
apiversion: V1
Kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: myapp
  ports:
  - protocol: TCP
    Port: 80
    target Port: 9376
```

Desire state → replica - 5 Desire

Actual state Matches the desire state. Actual

The Actual State can be running to satisfy the desire state

2nd spec for containers

Prad

Understanding Kube objects.

Kubernetes objects are persistent entities in Kubernetes system. Kubernetes use these entities to represent state of cluster.

- containerized application running
- resource avail to application
- Policies around application

Deployment resource object.

```
apiversion: apps/v1
Kind : Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2
  template:
    metadata:
```

```
spec:
  containers:
    - name: nginx
      image: nginx:1.14.2
      Port:
        - containerPort : 80
```

Service

Kubectl get svc /service name.

Kubectl delete svc name

Kubectl expose deployment /name

--type = NodePort --Port=80

exposing the port to the external world

Kubectl get svc/ services

X deployment is worker IP

Kubectl get Pods -o wide.

↓

deployment

↓

etc.

shows extra metadata

Selector app=name

Kubernets Accessing using

-> Declarative Method (Resource objects - YAML files)

-> Imperative Method.

(Kubernetes CLI)

Namespace is a seperate space in cluster.

Types of Services

- Cluster IP - Internal communication
- NodePort - Exposes service on node IP and Port
- Load Balancer - cloud Provider provide external load balancer
- External Name - Maps service to external DNS.

Role of Kube Proxy:

- Runs on every node
- Watches services and endpoints
- Creates iptables or IPVS rules
- Implements service virtual IP routing

IP: 10.244.1.219

Cmds:

- Kubectl version
  - ↳ Communicate to Kube API
  - ↳ Pass all cmds
  - To show version of client, server etc.
- Kubectl get nodes.
  - Show all nodes in cluster and status.
- Kubectl describe node nodename / pod podname.
  - ↳ like inspect in docker.
  - Metadata of node / pod name.
- Kubectl run podname --image imagename
  - create Pod.
- Kubectl exec -it podname -- /bin/bash
  - Or

Kubernetes Networking

Why Network?

- Containers are dynamic & short lived
- Pods can be create & destroed any time
- Applications must be communicate reliably
- Kubernetes provides a cluster-wide Network Model.

Networking Model.

-> Each Pod gets a unique IP address
-> Pods communicate directly without NAT
-> Nodes can communicate with Pods
-> Containers in same pod share the same network namespace.

CNI - Container Network Interface.

- CNI is standard used to configure networking for containers
- It assigns IP addresses to Pods.
- It connects Pods to the cluster Network
- Ex: calico, Flannel, Cilium.

Kubernetes Service:

- Service provides stable network endpoint for Pods.
- Pods are temporary and IP's changes
- Service provide a fixed IP and DNS name
- They load balance traffic across multiple Pods.

192 168 137 79